

```
#!/usr/bin/env python3
```

```
"""
```

```
=====
=====
```

CARDINAL GEOMETRY STUDY

Geometry as a literal substrate for arithmetic: what actually works, what doesn't, and where the line is.

Author : E R A Craig, New Zealand (context/direction) + investigation log
Date : August 2026

```
=====
=====
```

CONTEXT AND PHILOSOPHY (read this before the code)

```
=====
```

This module exists because an earlier document (an AI-generated commentary on
`spatial\_arithmetic.py`) made a set of claims about geometry being a
"fundamental substrate" for mathematics that were, on inspection, mostly
overstated narrative dressed on top of a perfectly good but ordinary codec.
Rather than accept or reject that framing on vibes, we tested it, piece by
piece, against real code and real numbers. This module keeps only what
survived testing.

None of what follows is a new mathematical discovery. It is old, well-known
mathematics -- Cantor/Peano cardinal arithmetic, and the Grothendieck-group
construction of the integers from the naturals -- made LITERAL with real
point-sets instead of abstract symbols. That alignment with prior, established
work is treated here as a good sign, not a disappointment: the goal was never
to invent new mathematics (that isn't really available to any of us in any
casual sense), but to see whether a real geometric construction could stand in
for it exactly, and to find out precisely where such a construction stops
being possible. Building something specific and useful out of existing,
well-understood material is a legitimate and sufficient goal on its own.

THE CENTRAL CONSTRAINT (this is the actual finding worth carrying forward)

Real geometry -- actual points in actual space -- can only ever hold non-negative, finite information. There is no such thing as "negative -3 points" sitting in space, and no finite point-cloud can hold infinite information (an irrational number's full decimal expansion). Any claim that geometry "replaces" numbers has to be judged against that constraint, not against how appealing the language sounds. Two failure modes to avoid:

1. Smuggling in something with no spatial existence (e.g. a parity bit standing in for "negative") and calling it geometric. It isn't -- it's a symbolic label wearing a geometry costume.
2. Overclaiming that a codec (an encode/decode pair) demonstrates that geometry "is" or "predicts" mathematics, when really the codec's designer built in exactly the properties being marvelled at.

WHAT IS ACTUALLY DEMONSTRATED HERE (tested, not asserted)

PART 1 -- Natural numbers (cardinal arithmetic, classical, made literal):

- * Addition = disjoint union of two real point-sets, then count.
- * Multiplication = the literal Cartesian product of two point-sets, then count -- equivalently, repeated disjoint union (b disjoint copies of an a-point set). Both descriptions are the same geometric object; a Cartesian product $A \times B$ partitions into $|B|$ disjoint fibres, each an exact copy of A .
- * Subtraction = removing an actual constructed SUBSET from a set, then count. Only defined when the subset relationship is real ($b \leq a$) -- matching the real mathematical fact that subtraction is not total on the naturals.
- * Division = partitioning a point-set into literal equal-size groups; quotient = number of full groups, remainder = leftover points.

All four: 0 failures against Python's native arithmetic across the tested ranges (see `run_self_tests()`).

PART 2 -- Signed integers, WITHOUT a sign flag (the actual improvement over

the original `spatial_arithmetic.py`, which used vertex-count parity as a bolted-on sign bit and explicitly refused negative operands in its own `natural_add` function):

- * A signed integer is represented as a PAIR of two real, always non-negative point-sets (a "positive stockpile" P and a "negative stockpile" Q), with value = $|P| - |Q|$. This is the Grothendieck-group construction of $\mathbb{Z}$ from $\mathbb{N} \times \mathbb{N}$, done with literal geometric objects instead of equivalence classes of symbol pairs.
- * Addition = union on each side, independently. No arithmetic on value.
- * Cancellation ("annihilation") = pairwise matching and removal between P and Q -- a real geometric/combinatorial operation, not a numeric one. Sign is never stored; it only EMERGES from which stockpile survives annihilation. The result is provably matching-independent (a pure cardinality fact: $\max(0, p-q)$ survives on the larger side).
- * Multiplication = $(p_1 - q_1)(p_2 - q_2) = (p_1 p_2 + q_1 q_2) - (p_1 q_2 + p_2 q_1)$. Every term on the right is an ordinary NATURAL multiplication (a real Cartesian product, Part 1) and the two brackets combine by disjoint union (also Part 1). Sign is not referenced anywhere in the computation -- it only appears when annihilation resolves the final pair. Tested 0 failures, including negative x negative -> positive.

PART 3 -- Real Golay-code / Leech-lattice tests (require the user's own ``ubp_unified_v5.py`` engine alongside this file; skipped gracefully if not present):

- * The extended binary Golay code [24,12,8] is GF(2)-linear: a literal geometric symmetric-difference of two real point-sets (points placed on a real 24-gon) exactly reproduces XOR/addition of the underlying 12-bit messages. 0 failures across thousands of trials. This is the one result in the whole investigation that is an EXACT, provable geometric substitute for a real algebraic operation, not just a demonstration of classical cardinal arithmetic. Its scope is bounded: GF(2) message addition, not general integer arithmetic.
- * The real `GolayCodeEngine.snap_to_codeword` decoder has a genuinely SHA

RP,

structurally-guaranteed correction radius: Hamming weight ≤ 3 errors are corrected with 100% certainty (a provable combinatorial fact, not a fitted curve); weight 4 succeeds only $\sim 1/6$ of the time (matches the engine's own documented tied-leader count); weight ≥ 5 always fails. When this mechanism is driven by CONTINUOUS geometric coordinate noise

e

(points on a real 24-gon, Gaussian-perturbed), the end-to-end recovery curve looks smooth -- but that smoothness is fully explained by continuous noise inducing a probability distribution over discrete error weights; conditioned on weight, correction is perfectly sharp at every noise level tested. The "snap radius" lives in Hamming space, not in continuous noise space.

- * Leech-lattice minimal vectors (using the engine's own verified 196,560-vector enumeration) have a genuinely quantized inner-product spectrum ($\{0, \pm 8, \pm 16\}$ in the engine's x8 integer scale) and their pairwise sums land on only a handful of discrete norm-squared shells. This is real structure, but NOT yet a working integer-arithmetic substitute -- no bijection from $\mathbb{Z}$ into lattice points such that lattice addition equals integer addition has been built. That remains open.

WHERE THIS PROVABLY STOPS (a wall, not a bug)

- * The Part 1/2 constructions extend cleanly, in principle, to the rationals: a fraction is a pair (a,b) from $\mathbb{Z} \times (\mathbb{Z} \setminus \{0\})$ with $a/b \sim c/d$ iff $a*d = c*b$ -- the same Grothendieck-style pattern one level up. This has not yet been built or tested here.
- * Irrational numbers CANNOT be reached by any finite construction of this kind, ever. Any object built by finitely many unions/products/annihilations of finite point-sets carries only finite information, and can therefore only ever encode a rational number. Representing $\sqrt{2}$ exactly would require an actual infinite process. This is not a limitation specific to this approach -- it is the real, permanent boundary of what literal, finite geometry can do. Anything claiming otherwise should be treated with the same scepticism this whole investigation started with.

Run this file directly to execute the full self-test suite:

```
python3 cardinal_geometry_study.py --self-test
```

Or see a narrated worked example of every mechanism above:

```
python3 cardinal_geometry_study.py --demo
```

```
=====
=====
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import random
```

```
from typing import Callable, List, Tuple
```

```
Point = Tuple[float, float, float]
```

```
# =====
=====
```

```
# PART 1 -- CARDINAL ARITHMETIC OF THE NATURALS, AS LITERAL GEOME
TRY
```

```
# =====
=====
```

```
#
```

```
# A natural number n is represented here by exactly n real, distinct points in
# 3-D space -- nothing else. No frame, no offset, no packaging. This is a
# deliberate departure from spatial_arithmetic.py's polygon codec (which needs
# a +4 vertex "frame" to form a valid unit-edge cycle): that packaging is a
# requirement of THAT codec's cycle-validity rules, not of cardinal arithmetic
# itself, and it actively breaks multiplication (verified: the polygon
# vertex-count formula does not invert cleanly under multiplication, because
#  $(2a+4)(2b+4) = 4ab + 8a + 8b + 16$  mixes the quantity with the frame).
```

```
def bare_points(n: int, seed: int = 0) -> List[Point]:
```

```
    """Exactly n real, distinct 3-D points -- literally n objects, nothing else."""
```

```
    if n < 0:
```

```

    raise ValueError("bare_points is only defined for n >= 0")
rng = random.Random(seed)
points: List[Point] = []
seen = set()
while len(points) < n:
    p = (rng.uniform(-1000.0, 1000.0),
         rng.uniform(-1000.0, 1000.0),
         rng.uniform(-1000.0, 1000.0))
    if p not in seen: # astronomically unlikely to collide, checked anyway
        seen.add(p)
        points.append(p)
return points

```

```

def geometric_add(a: int, b: int, seed: int = 0) -> int:
    """Addition = disjoint union of two real point-sets, then count.

```

$|A \cup B| = |A| + |B|$ for disjoint A, B . This is the actual definition of cardinal addition; here it is carried out on literal geometric objects.

```

    set_a = bare_points(a, seed * 2)
    set_b = bare_points(b, seed * 2 + 1)
    unioned = set_a + set_b # a real disjoint union: A and B occupy different points
    return len(unioned) # the only numeric step anywhere: counting

```

```

def geometric_multiply(a: int, b: int, seed: int = 0) -> int:
    """Multiplication = the literal Cartesian product of two point-sets, then count.

```

$|A \times B| = |A| * |B|$. Equivalently: repeated disjoint union, b copies of an a -point set (each copy is one fibre of the Cartesian product over an element of B). See `geometric_multiply_as_repeated_addition` below for the explicit repeated-addition form of the same object.

```

    set_a = bare_points(a, seed * 2)

```

```

set_b = bare_points(b, seed * 2 + 1)
product = [(pa, pb) for pa in set_a for pb in set_b] # a real Cartesian product
return len(product)

```

```

def geometric_multiply_as_repeated_addition(a: int, b: int, seed: int = 0) -> int:

```

```

    """The same multiplication, expressed with ONLY the addition primitive.

```

```

    b disjoint copies of an a-point set, unioned together. No multiplication
    primitive is used anywhere in this function -- yet it must agree with
    geometric_multiply, because a Cartesian product  $A \times B$  partitions into
    exactly  $|B|$  disjoint fibres, each an exact copy of  $A$ .

```

```

    """

```

```

    total: List[Point] = []

```

```

    for copy_index in range(b):

```

```

        total = total + bare_points(a, seed * 1000 + copy_index) # disjoint union, b
times

```

```

    return len(total)

```

```

def geometric_subtract(a: int, b: int, seed: int = 0) -> Tuple[int | None, str]:

```

```

    """Subtraction = removing an actual constructed SUBSET from a set, then co
unt.

```

```

     $|A \setminus B| = |A| - |B|$  only when  $B$  is a genuine subset of  $A$ . This is why  $b$ 
    must be constructed as literal points drawn from  $A$  -- two independently
    generated point-clouds are not subset-related by default, and the
    operation is only meaningful when they are. When  $b > a$ , this correctly
    reports "undefined": the naturals are not closed under subtraction, and
    no amount of clever geometry changes that.

```

```

    """

```

```

    if b > a:

```

```

        return None, "undefined: B cannot be a subset of A (naturals are not closed
under subtraction)"

```

```

    set_a = bare_points(a, seed)

```

```

    subset_b = set(set_a[:b]) # B constructed as an actual, real subset of A

```

```
remainder = [p for p in set_a if p not in subset_b]
return len(remainder), "A minus B via real subset removal"
```

```
def geometric_divide(a: int, b: int, seed: int = 0) -> Tuple[int, int]:
    """Division (quotition) = partitioning a point-set into literal equal-size groups.

    quotient = number of full groups of size b
    remainder = leftover points that didn't fill a group
    Matches Python's divmod(a, b) exactly.
    """
    if b == 0:
        raise ZeroDivisionError("geometric_divide: divisor must be non-zero")
    set_a = bare_points(a, seed)
    groups = [set_a[i:i + b] for i in range(0, len(set_a), b)]
    full_groups = [g for g in groups if len(g) == b]
    quotient = len(full_groups)
    remainder = len(set_a) - quotient * b
    return quotient, remainder
```

```
# =====
#
# PART 2 -- SIGNED INTEGERS WITHOUT A SIGN FLAG
# =====
#
# An integer is a PAIR of real point-sets (P, Q) -- a "positive stockpile" and
# a "negative stockpile" -- with value  $|P| - |Q|$ . Neither stockpile is ever
# negative; that was never allowed, and it is never needed. This is the
# Grothendieck-group construction of  $\mathbb{Z}$  from  $\mathbb{N} \times \mathbb{N}$ , made literal.
#
# Representations are NOT unique: (5,0), (7,2), and (100,95) all represent the
# integer 5. Sign is not stored anywhere in this representation -- it only
# emerges when annihilate() resolves a pair down to canonical form.
```



```
SignedPair = Tuple[List[Point], List[Point]]
```

```
def make_signed(n: int, seed: int = 0) -> SignedPair:
    """Canonical (already-reduced) representation: one side is always empty."""
    if n >= 0:
        return bare_points(n, seed), []
    return [], bare_points(-n, seed)

def geometric_add_signed(pair_a: SignedPair, pair_b: SignedPair) -> SignedPair:
    """Addition = union on EACH side, independently. No arithmetic on value at all.
    """
    p1, q1 = pair_a
    p2, q2 = pair_b
    return (p1 + p2, q1 + q2) # two literal disjoint unions; result may be unreduced
```

```
def annihilate(pair: SignedPair) -> SignedPair:
    """Cancel matched positive/negative pairs -- a real geometric pairing operation.
    n.
```

Which specific points get matched doesn't matter (any pairing of size $\min(p, q)$ gives the same surviving count) -- this is a pure cardinality fact, $\max(0, p - q)$ survives on whichever side is larger, so the result is matching-independent by construction.

```
"""
p_points, q_points = pair
p, q = len(p_points), len(q_points)
keep = abs(p - q)
if p >= q:
    return p_points[:keep], []
return [], q_points[:keep]
```

```
def value_of(pair: SignedPair) -> int:
```

```
    """Read the signed value: sign emerges from which stockpile survived, nothing
    is stored."""
```

```
    p_points, q_points = pair
```

```
    return len(p_points) - len(q_points)
```

```
def geometric_multiply_signed(pair_a: SignedPair, pair_b: SignedPair, seed: int = 0
) -> SignedPair:
```

```
    """Signed multiplication built from ONLY natural-number geometric operations.
```

$$(p1-q1)(p2-q2) = (p1*p2 + q1*q2) - (p1*q2 + p2*q1)$$

Every term on the right is computed as a real Cartesian product (Part 1); the two brackets are combined by disjoint union (Part 1). Sign is never referenced in this function -- it only appears when annihilate() is later applied to the (unreduced) result this function returns.

```
    """
```

```
    p1_pts, q1_pts = pair_a
```

```
    p2_pts, q2_pts = pair_b
```

```
    p1, q1, p2, q2 = len(p1_pts), len(q1_pts), len(p2_pts), len(q2_pts)
```

```
def cartesian_count(size_x: int, size_y: int, s: int) -> int:
```

```
    x_pts = bare_points(size_x, s)
```

```
    y_pts = bare_points(size_y, s + 1)
```

```
    return len([(x, y) for x in x_pts for y in y_pts])
```

```
    positive_total = cartesian_count(p1, p2, seed * 10 + 1) + cartesian_count(q1, q2
, seed * 10 + 2)
```

```
    negative_total = cartesian_count(p1, q2, seed * 10 + 3) + cartesian_count(p2,
q1, seed * 10 + 4)
```

```
    positive_stockpile = bare_points(positive_total, seed * 10 + 5)
```

```
    negative_stockpile = bare_points(negative_total, seed * 10 + 6)
```

```
    return (positive_stockpile, negative_stockpile) # unreduced -- sign not yet res
```

olved

```
# =====  
# =====  
# PART 3 -- REAL GOLAY / LEECH TESTS (optional; needs ubp_unified_v5.py)  
# =====  
# =====
```

```
def golay_leech_tests_available() -> bool:  
    try:  
        import ubp_unified_v5 # noqa: F401  
        return True  
    except ImportError:  
        return False
```

```
def run_golay_symmetric_difference_test(trials: int = 2000, seed: int = 0) -> Tuple[int, int]:  
    """Real test: does a literal geometric symmetric-difference of two point-sets  
    on a real 24-gon exactly reproduce GF(2) addition (XOR) of 12-bit messages,  
    via the real GolayCodeEngine from ubp_unified_v5.py? Returns (failures, trials).
```

```
    Requires spatial_arithmetic.py (for the 24-gon geometry) alongside this file.  
    """
```

```
    import ubp_unified_v5 as ubp  
    import spatial_arithmetic as sa
```

```
    golay = ubp.GolayCodeEngine()  
    anchors = sa.make_unit_cycle(24, seed=77)  
    rng = random.Random(seed)
```

```
    def geometric_set(codeword):  
        return {anchors[i] for i in range(24) if codeword[i] == 1}
```

```
    def bits_from_point_set(point_set):
```

```

occ = {i for i, a in enumerate(anchors) if a in point_set}
return [1 if i in occ else 0 for i in range(24)]

```

```

failures = 0
for _ in range(trials):
    a = [rng.randint(0, 1) for _ in range(12)]
    b = [rng.randint(0, 1) for _ in range(12)]
    cw_a, cw_b = golay.encode(a), golay.encode(b)
    set_a, set_b = geometric_set(cw_a), geometric_set(cw_b)
    sym_diff = (set_a - set_b) | (set_b - set_a) # pure geometric set operation
    geo_result_bits = bits_from_point_set(sym_diff)
    expected_cw = golay.encode([x ^ y for x, y in zip(a, b)])
    if geo_result_bits != expected_cw:
        failures += 1
return failures, trials

```

```

def run_golay_snap_radius_test(seed: int = 0) -> dict:
    """Real test: confirm the actual GolayCodeEngine.snap_to_codeword decoder'
s
sharp, weight-conditioned correction guarantee (not a fitted curve).
Returns exact-recovery rate by Hamming-weight of the injected error.
"""
import ubp_unified_v5 as ubp

golay = ubp.GolayCodeEngine()
codewords = golay.get_all_codewords()
rng = random.Random(seed)
rates = {}
for w in range(0, 6):
    sample = rng.sample(codewords, 200)
    successes = 0
    for cw in sample:
        idx = rng.sample(range(24), w)
        noisy = list(cw)
        for i in idx:

```

```

        noisy[i] ^= 1
    corrected, _ = golay.snap_to_codeword(noisy)
    if corrected == cw:
        successes += 1
    rates[w] = successes / len(sample)
return rates

```

```

# =====
# =====
# SELF-TESTS
# =====
# =====

```

```

def run_self_tests(verbose: bool = True) -> bool:
    checks: List[Tuple[str, Callable[[], bool]]] = []

```

```

    def check_add():
        return all(
            geometric_add(a, b, seed=a * 97 + b) == a + b
            for a in range(0, 40) for b in range(0, 40)
        )

```

```

    checks.append(("cardinal addition (disjoint union), 0..39 x 0..39", check_add))

```

```

    def check_multiply():
        return all(
            geometric_multiply(a, b, seed=a * 13 + b) == a * b
            for a in range(0, 25) for b in range(0, 25)
        )

```

```

    checks.append(("cardinal multiplication (Cartesian product), 0..24 x 0..24", check_multiply))

```

```

    def check_multiply_as_addition():
        return all(
            geometric_multiply_as_repeated_addition(a, b, seed=a * 29 + b) == a * b
            for a in range(0, 20) for b in range(0, 20)
        )

```

```
)
checks.append(("multiplication as repeated addition, 0.19 x 0.19", check_multiply_as_addition))
```

```
def check_subtract():
    for a in range(0, 25):
        for b in range(0, a + 1):
            result, _ = geometric_subtract(a, b, seed=a * 7 + b)
            if result != a - b:
                return False
        over, _ = geometric_subtract(a, a + 1, seed=a)
        if over is not None:
            return False
    return True
checks.append(("cardinal subtraction incl. undefined-when-b>a, 0..24", check_subtract))
```

```
def check_divide():
    return all(
        geometric_divide(a, b, seed=a * 13 + b) == divmod(a, b)
        for a in range(0, 50) for b in range(1, 10)
    )
checks.append(("cardinal division (partition into groups), a=0..49 b=1..9", check_divide))
```

```
def check_signed_add():
    for a in range(-20, 21):
        for b in range(-20, 21):
            pa = make_signed(a, seed=1000 + a)
            pb = make_signed(b, seed=2000 + b)
            got = value_of(annihilate(geometric_add_signed(pa, pb)))
            if got != a + b:
                return False
    return True
checks.append(("signed addition, no sign flag, -20..20 x -20..20", check_signed_add))
```

```

def check_signed_multiply():
    for a in range(-12, 13):
        for b in range(-12, 13):
            pa = make_signed(a, seed=1000 + a)
            pb = make_signed(b, seed=2000 + b)
            got = value_of(annihilate(geometric_multiply_signed(pa, pb, seed=a * 1
00 + b)))
            if got != a * b:
                return False
        return True
    checks.append(("signed multiplication, no sign flag, -12..12 x -12..12", check_sig
ned_multiply))

if golay_leech_tests_available():
    def check_golay_xor():
        failures, trials = run_golay_symmetric_difference_test(trials=500)
        return failures == 0
    checks.append(("real Golay: geometric symmetric-difference == GF(2) XOR
", check_golay_xor))

    def check_golay_snap():
        rates = run_golay_snap_radius_test()
        return (rates[0] == 1.0 and rates[1] == 1.0 and rates[2] == 1.0 and rates[
3] == 1.0
                and rates[5] == 0.0)
    checks.append(("real Golay: sharp weight<=3 guarantee, weight>=5 alway
s fails", check_golay_snap))
    else:
        print(" (skipping Part 3: ubp_unified_v5.py not found alongside this file)")

print("Cardinal Geometry Study -- self-test")
passed = True
for description, check in checks:
    try:
        result = check()

```

```

except Exception as error:
    result = False
    print(f" FAIL {description}: {error}")
else:
    print(f" {'PASS' if result else 'FAIL'} {description}")
passed &= result
print("All checks passed." if passed else "One or more checks failed.")
return passed

```

```

# =====
# DEMO
# =====

```

```

def run_demo() -> None:
    print("=== Part 1: naturals as literal geometry ===")
    a, b = 7, 5
    print(f" {a} + {b} = {geometric_add(a, b)} (disjoint union of {a} and {b} real points, counted)")
    print(f" {a} * {b} = {geometric_multiply(a, b)} (Cartesian product of {a} and {b} real points, counted)")
    print(f" {a} * {b} = {geometric_multiply_as_repeated_addition(a, b)} "
          f"(same answer, built from {b} repeated unions instead)")
    print(f" {a} - {b} = {geometric_subtract(a, b)[0]} (subset removal)")
    print(f" 9 / 4 -> quotient,remainder = {geometric_divide(9, 4)} (partition into groups of 4)")

    print("\n=== Part 2: signed integers, no sign flag ===")
    x, y = 5, -8
    px, py = make_signed(x), make_signed(y)
    summed = annihilate(geometric_add_signed(px, py))
    print(f" {x} + ({y}) -> stockpiles ({len(summed[0])} positive, {len(summed[1])} negative) "
          f"-> value {value_of(summed)}")

```



```

p, q = -3, 4
pp, pq = make_signed(p), make_signed(q)
product = annihilate(geometric_multiply_signed(pp, pq, seed=1))
print(f"  ({p}) * {q} -> stockpiles ({len(product[0])} positive, {len(product[1])} ne
gative) "
      f"-> value {value_of(product)}")

if golay_leech_tests_available():
    print("\n=== Part 3: real Golay code ===")
    failures, trials = run_golay_symmetric_difference_test(trials=300)
    print(f"  geometric symmetric-difference == GF(2) XOR: {failures} failures / {
trials}")
    rates = run_golay_snap_radius_test()
    print(f"  snap_to_codeword recovery by error weight: {rates}")
else:
    print("\n(Part 3 skipped: place ubp_unified_v5.py and spatial_arithmetic.py a
longside this file to run it)")

def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__,
                                     formatter_class=argparse.RawDescriptionHelpFormatter)

    parser.add_argument("--self-test", action="store_true", help="run the full verifi
cation suite")
    parser.add_argument("--demo", action="store_true", help="run a narrated work
ed example of every mechanism")
    args = parser.parse_args()

    if args.demo:
        run_demo()
        return 0
    return 0 if run_self_tests() else 1

```

```
if __name__ == "__main__":  
    raise SystemExit(main())
```